



University of Bahrain
College of Information Technology
Department of Computer Science
ITCY354 – Secure Software Engineering

From Requirements to Design: Building Secure Software Models

SecureFocus: A Secure Productivity and Collaboration Suite

Team Members:

Noora Abdulrahman Bumajaid-202203245

Manar Hussain Abdulrahim-202201554

Raghad Abdulla-202109753

Submitted on:30/11/2025

SEC:01

Table Of Contents

1. Problem Definition.....	4
2. Project Objectives.....	5
3. Significance and Impact.....	6
4. Brief Literature Review.....	7
5. Scope and Limitations.....	9
5.1 Project Scope.....	9
5.2 What is Excluded.....	10
5.3 Constraints and Assumptions.....	10
6. Software Process Model.....	11
7. Project Management Plan.....	14
7.1 Timeline & Milestones.....	14
7.2 Team Roles & Responsibilities.....	15
7.3 Communication Methods.....	16
8. Risk Management Plan.....	17
Table1: Summary of project risks, their likelihood, effects, and the mitigation strategies applied....	17
9. Functional and Non-Functional Requirements.....	19
9.1 Functional Requirements.....	19
9.2 Non-Functional Requirements.....	21
10. Security Requirements (Using the SQUARE Methodology).....	22
Table2: List of identified security risks with their impact, likelihood, and priority levels.....	24
Table3: Security requirements categorized by priority and traceability.....	26
11. System and Design Modeling.....	27
11.1 High-Level System Architecture.....	27
11.2 Use Case Diagram.....	27
Figure1: Use case diagram showing user and admin interactions with the system.....	29
11.3 Class Diagram.....	29
11.4 Data Flow Diagram (DFD):.....	32
Figure3: Level-0 DFD of SecureFocus showing user interactions and data flows.....	32
Figure4: Level-1 DFD detailing the internal processes and data interactions within the SecureFocus system.....	34
11.5 Interface Design: Figma.....	35
Figure5: SecureFocus login interface showing authentication fields, role options, and security	

indicators.....	35
Figure6: Standard user dashboard displaying tasks organized by status and project workflow...	36
Figure7: Task creation window for adding a new task with details, priority, assignee, and due date.	37
Figure8: SecureFocus messaging interface showing encrypted conversations and chat threads....	38
Figure9: Secure file upload interface showing encrypted file uploads and stored files.....	38
Figure10: Admin dashboard displaying system health, user statistics, and key administrative actions.....	39
Figure11: Admin audit log panel showing detailed system activity records with filtering options....	40
Figure12: Admin user management panel showing user accounts, roles, statuses, and login activity.....	40
Figure13: Admin form for creating a new user account with name, email, and role selection.....	41
Figure14: System configuration panel with security and project settings.....	41
11.6 Secure Architectural Principles.....	42
12. Threat Modeling and Security Analysis.....	42
12.1 Overview of STRIDE Categories.....	43
Table4: STRIDE threat categories with their meanings and the security objectives they violate..	43
12.2 Threat Identification and Analysis.....	44
Table5: Threat analysis table outlining STRIDE threats, impacts, and proposed mitigations.....	44
12.3 Threat Prioritization.....	46
12.4 Summary.....	47
13. Evaluation and Discussion.....	47
13.1 Trade-Offs.....	47
13.2 Challenges Faced.....	48
13.3 Lessons Learned.....	49
14. Future Work.....	50
14.1 System Implementation and Deployment.....	50
14.2 Enhanced Security Validation.....	51
14.3 Advanced Features and Functionality.....	51
15. References.....	52

1. Problem Definition

SecureFocus came from a problem that's getting worse but is rarely talked about. Most teamwork apps don't have strong security built in from the start. With more people working remotely and relying on cloud systems, companies now use digital tools daily for things like messaging, sharing files, managing projects, or chatting internally (Stevens, 2025). Even though these platforms speed up collaboration, they also create chances for hackers to steal data, invade privacy, or sneak into sensitive areas without permission.

The truth? Lots of current platforms care more about ease of use, along with smooth connections, than actual safety. If security gets tacked on after launch instead of being designed up front, weak spots usually show up sooner or later. Through the years, major leaks tied to well-known teamwork apps have proven just how shaky online work environments can turn out when proper defenses aren't active. These events made it clear there's no skipping solid encryption, layered login checks, or tight permission rules if both people and companies want real protection.

This leads straight to the main problem: how do teams stay linked, swap thoughts, yet keep sensitive info safe? SecureFocus tackles it by mixing safety with a smooth workflow - all built into one unified setup. It weaves together private messaging with encryption from start to finish, locked file storage, along with multi-step login checks across one single hub - keeping every chat and action shielded and traceable.

From a safety-minded tech angle, SecureFocus grabs attention by stopping threats before they start. Instead of waiting, it blocks data spills, fake identities, and hacking moves right at the core design. This ahead-of-the-game mindset boosts user confidence while raising the bar for teamwork apps, making them fast, smooth, yet rock solid in protection.

2. Project Objectives

The main goals of SecureFocus are straightforward, trackable, and backed by trusted software security practices. Each ties straight into key security aspects, keeping data private, accurate, and accessible, plus verifying identity and access rights. As a whole, they shape how the system runs, holds up, and keeps things safe.

O1: Private teamwork: First up, build a website for team use where every bit of info stays locked down with top-tier encryption like AES-256 and TLS 1.3, whether sitting still or moving across networks. That way, if someone grabs the data mid-transfer, it won't make sense to them. To check if this works, run real-life tests, like mimicking data eavesdropping to make sure no private details slip out when things get messy.

O2: Strong login checks: SecureFocus will use multi-step verification along with access rules based on user roles. Access rights rely on confirmed identity combined with job function. The goal is to shut down unapproved entry attempts, tested thoroughly using simulated attacks, when fake logins or identity tricks don't work.

O3: Safe Messaging and Sending Files: This system lets people chat and share documents securely through encryption. Messages and transfers are locked from end to end, so SecureFocus servers can't peek at any of it. To confirm safety, we'll make sure encryption keys stay on user devices while testing with fake man-in-the-middle attacks to see if privacy stays strong.

O4: Integrity plus Non-Repudiation: Each key action, like changing files or updating project tasks, gets logged in an audit trail that shows if someone messed with it. If data changes while moving, digital signatures or hash checks will spot it right away. That way, there's clear responsibility that anyone can see who did what and at what time.

O5: Staying Up and Running: SecureFocus will keep working even when things get rough or someone tries to crash it. Goals to stay live more than 99% of the time when used normally, while slowing down smoothly instead of failing hard during traffic spikes or DoS attacks. Testing with heavy loads, along with mapping out threats, checks whether it can handle stress without breaking down.

Once these goals are met, SecureFocus isn't just another tool for getting things done. Instead, it works like a system built with security first, shielding information, stopping abuse before it starts, while maintaining performance no matter the stress.

3. Significance and Impact

SecureFocus works for groups dealing with private information those needing teamwork tools without risking exposure. Small firms nervous about spills use it, just as distant crews wanting safe chats do. Think clinics, law offices, banks - places where rules like GDPR or HIPAA push tighter locks on data. Since safety's the main thing here, people skipping regular cloud apps often switch when they want more control.

The project gives you a set of work tools that come with safety baked right in, not tacked on later. Instead of worrying about leaks or hacks, people handle jobs, plans, and chats knowing data stays locked down. Different from usual apps, each time someone uploads a file in SecureFocus, it gets scrambled on their device first , so even staff running the system can't peek unless allowed.

With this no-assumptions setup, users feel safer using shared tools instead of falling back on risky old habits like sending files through regular email. When tools get better, things like data protection, multi-step logins, or rule-following aren't optional anymore. With SecureFocus, safe coding doesn't mean awkward workflows - both work fine together, which makes online setups feel more reliable again.

In terms of school study, SecureFocus shows solid methods in building safe software - using systems like SQUARE and STRIDE within an actual development environment. Instead of waiting till later phases, it proves how safety checks fit right at the start of creating software. What stands out is how protection works not just as a tech fix but also boosts company value.

Because it tackles risks such as leaked info or fake user access common in teamwork apps, this tool cuts down on lost business data. More than just another app, it pushes forward the idea that secure design should shape team tools from day one, making safety central rather than something added after.

4. Brief Literature Review

Nowadays plenty of teamwork and efficiency apps already exist. Among them Google Workspace plus Microsoft 365 stand out - they bundle services such as messaging, schedules, file work, along with instant talk inside one online space. While these packs pack lots of features, people have worried about privacy for years because firms could see user info if it's not encrypted on the device first.

Lately though both giants improved safety tools; take Google's new option that encrypts Drive plus Gmail files right from your own machine. Then there's another type - open alternatives including Nextcloud - that you can run yourself while enabling full encryption by default. Nextcloud runs on a system where the server can't peek at your files or passwords - everything stays locked up tight. Because of this setup, your info's way safer, but getting it running might mean wrestling with tricky configs; plus, you won't find all the slick extras that come with apps from Google or Microsoft.

Compared to Google Workspace or Microsoft 365, SecureFocus does less overall - yet locks things down way tighter. While those big-name options work smoothly and let people edit

together live, they generally protect data just while it's moving or sitting on servers. Unless you turn on extra settings, Google and Microsoft keep the keys needed to unlock your info.

With SecureFocus, though, every file and message gets locked using encryption right on your device - and that's turned on from day one. Unlike patchwork setups like Nextcloud, this system bakes in teamwork features - think shared task lists and messaging - all inside one protected space. Though tools such as Mattermost or Rocket.Chat let teams message securely while hosting on their own servers, they miss blending chat with document exchange and project tracking into a unified system. SecureFocus wants to bring those pieces together - wrapped in strong privacy from the start.

Some teamwork apps today force people to pick either ease or secrecy. Take Google Drive - turn on device-level scrambling, and you can't edit together live since locked files won't work on their systems. With Nextcloud, full encryption used to run only in set directories, plus it brought headaches managing keys. SecureFocus aims to fix these gaps using a smarter layout: things like chats and documents stays tightly sealed from end to end, whereas info such as project notes gets solid protection when saved, though not always fully scrambled throughout, so tools like alerts and lookup still function smoothly.

Looking at what's already out there, it's obvious we're missing a single tool that teams can use together - one built with strong privacy from the start but doesn't slow things down. SecureFocus wants to take what works well now - smooth teamwork features, solid data protection, plus the chance to run your own server - and fix where others fall short when it comes to safety and simplicity.

5. Scope and Limitations

5.1 Project Scope

SecureFocus aims to provide all the necessary elements needed in terms of achieving both security and efficiency in collaboration. SecureFocus offers all basic modules in one web interface.

(a) User Management:

This will involve protected user registration, login, as well as user profile management with the help of Multi-Factor Authentication and Role-Based Access Control. This will involve the use of roles like Regular User, Admin, among others.

(b) Project & Task Management:

Users will be able to create projects, add and assign tasks, create due dates, and follow progress. In addition to allowing users to view their workloads visually.

(c) Secure Messaging:

There will be an in-built messaging module, which will facilitate communication on projects via encrypted text messages and file transfer. Each message sent in the chat will be encrypted, meaning even administrators cannot see the contents of the message.

(d) File Storage and Sharing:

SecureFocus will comprise secure file storage, which will be project space-specific. Users will be able to upload their files to the platform, which will be encrypted on the client side before reaching the server. Users will be able to share their encrypted files easily with other members in their project.

(e) Notifications and Calendar:

There will be an alert notification service, which notifies users of impending deadlines, updates, or status changes. Furthermore, there will be a basic calendar view, showing task deadlines and milestones. This service will be kept basic due to time limitations.

All of these will be integrated into one web interface, which can be accessed directly via modern browsers, without having to install anything else.

5.2 What is Excluded

This means that a scope of work that entails advanced functionalities, like the collaborative editing of documents in a browser-based environment, is not taken into consideration in this current scope of work. For instance, a scope of work that entails the collaboration of reports or spreadsheets in a browser-based environment is not something that SecureFocus aims to accomplish.

Video-conferencing facilities as well as voice conferencing are not available in this release. SecureFocus will only concentrate on text-based communication as well as transferring files. Moreover, although we will design and create prototypes of the security system, including cryptographic processes and UI designs, we shall not implement the cryptographic servers and production-level environment. Indeed, our security system will be at the design and prototyping stage. Also, Intrusion Detection Systems (IDS) and Incident Response Systems will not be included, since they represent part of the overall security environment.

5.3 Constraints and Assumptions

One assumption we make is that SecureFocus is able to handle medium-sized teams, probably in the several hundred-user range per organization level. Scalability to thousands of users is not in our immediate plans, but scaling should be possible due to our design, such as stateless web layers.

There's also an assumption made concerning how users handle their keys. And in our design, we'll be assuming that keys can be generated from the users' passwords or be locally resident in browsers. We wouldn't be including more complex solutions such as Hardware Security Modules (HSMs) in our design.

Having a stable and trustworthy network environment is another assumption. The existence of end-to-end encryption involves dependencies on the availability of the network, and our design assumption will be that the key exchange protocols work correctly.

And, ultimately, due to time limitations, some of the security elements will be retained in design form rather than being fully functional. This means, in our instance, we will show how MFA works, with data flows, but we will not implement an MFA service utilizing, for example, emails or texts.

Nonetheless, the SecureFocus scope is complete in terms of secure engineering design, in that the scope covers all aspects of analysis, modeling, and design in relation to designing and verifying the structure of such a security-minded product.

6. Software Process Model

We had distinguished various models of the Software Development Life Cycle (SDLC) before choosing the Incremental (Iterative) model for SecureFocus. We reached this conclusion after weighing multiple alternatives on their treatment of changing requirements, security, and feedback during software development.

1- Waterfall Model:

The Waterfall approach has a rigid, linear pattern from requirement gathering, moving to design, then implementation, testing, and installation. This approach is organized, but not flexible. When

one stage is accomplished, it is difficult and costly to revert to the previous stage (“Introduction to Software Engineering,” 2011, 29). This is particularly a problem in our SecureFocus project. Our understanding of security needs will evolve along with the project, and the interests of our target users might change. We might realize we need another auditing component or simply need our current access validation to be a more robust partway into our project. Because our Waterfall approach does not adapt easily to change, we could not use it in our project (“Introduction to Software Engineering,” 2011, 29).

2- Agile Model:

The Agile approach emphasizes flexibility and adaptability. The Agile model promotes short development cycles, code reviews, and continuous integration. There is also frequent reassessment of requirements, and small, rapid changes are made rather than large, occasional changes. This Agile approach ensures quick development, which helps in the early testing of ideas, the removal of bugs quickly, and adjustment according to new understandings with minimal disturbances. This is applicable in our project, which might see changing technology and security priorities unexpectedly (Efimova, 2024).

3- Incremental Model:

The Incremental approach decomposes the task into smaller, more manageable chunks, each one of which adds to the overall functionality. We start with basic aspects such as task management and accounts, followed by secure messaging, storage, and other levels of security in further increments. Each increment gets more complex, with continuous build verification for detecting bugs or security vulnerabilities before they spread (Efimova, 2024). However, although such approaches call for highly effective risk management in every iteration (Efimova, 2024), they work much more effectively when adapting to changing security requirements.

4- Spiral Model:

We also looked at the Spiral model, which combines iterative development with risk analysis. This was useful for our project, which involves extensive security analysis, as every phase of the spiral involves risk analysis. However, we realized that the spiral model might be too heavy for our semester project, as every phase involves heavy documentation and analysis. Rather, we intend to incorporate the risk analysis phase of the spiral model in our Incremental approach.

Therefore, we decided on Incremental Development:

We will work with the Incremental model, integrating Agile principles into our workflow. Our development process will consist of three primary iterations:

Iteration 1:Build the foundation accounts, task management. Security will be minimal (simple auth only). The intent at this stage is to create a quick prototype, then harvest feedback with the prototype.

Iteration 2:Incorporate security. In this iteration, the SQUARE process will enhance security requirements, along with the implementation of encrypted file storage, secure messaging, and other security features. STRIDE threat modeling will be used on the baseline of the first iteration.

Iteration 3:Strengthen and Refine. This iteration will work on enhancing security architecture, enhancing user experience, specifically with regard to MFA, and completing full-scale testing.

Thus, every stage provides an implementable version to be tested. This makes way for improvements in both functionality and security simultaneously. The Incremental approach provides SecureFocus with exactly what we need. The required flexibility, validation, and the ability to adapt according to new findings. The Waterfall approach, with its rigid strategy, would be unable to keep up with the changes in the environment and security threats (“Introduction to Software Engineering,” 2011, 32).

7. Project Management Plan

7.1 Timeline & Milestones

The project took roughly ten weeks to complete, moving step by step along the class schedule.

Week 1–2: Project Initiation

In the opening two weeks, members came together while picking a focus - then got the green light. Plus, we pinned down what needed fixing and sketched out how far things would go.

Milestone: The project plan was sent in - then got the green light.

Week 3–5: Requirements Analysis

We collected what the system needs to do, such as handling tasks along with how users move through it. Meanwhile, the group began using SQUARE to spot and rank safety needs. Rough layout ideas plus scenarios got drawn up so we could see how everything fits together.

Milestone: The specs doc got finished.

Week 6–8: System Design & Prototype 1

The setup got sketched out, while a rough version came together featuring profile access plus job tracking tools. At that point, risks were checked over alongside STRIDE-style attack planning tied to the early blueprint.

Milestone: First working model done - plus DFD and Class Diagram sorted out.

Week 9: Security Integration & Prototype 2

This stage was about adding safety features directly into the setup. Because we tested core encoding actions, the SQUARE ranking list got locked in following team feedback. After that, various parts came together through the updated test version.

Milestone: Prototype two's done, featuring safe-mode test runs.

Week 10–11: Testing & Finalization

The crew ran functional tests along with security checks based on set scenarios. After reviewing the outcomes, trade-offs in design, plus general observations, everything got pulled together into the last version of the report.

Milestone: The final report was handed in – project showcased by Week 12.

7.2 Team Roles & Responsibilities

The trio split up jobs according to what each person was good at or cared about, though they stayed in tight contact during the whole task.

Noora – Project Lead & Documentation Coordinator

Noora kicked off the paperwork by outlining the issue, goals, and how far the project would go. Besides that, she looked into why the step-by-step method made sense, then built the tech setup and roadmap the group followed.

Manar – System Designer & Risk Planner

Manar shaped the system's big-picture design while also drafting up the risk log and strategies to handle potential issues. Alongside that, she pitched in on reviewing existing research plus took charge of timelines, organizing resources, and keeping everyone on the same page.

Raghad – Security Analyst & Evaluator

Raghad handled every security task tied to the project. Because she ran the SQUARE method, she also guided the STRIDE threat analysis. After that, she wrote the parts on assessment along with what should come next. Thanks to her efforts, security wasn't added later - it shaped the design from the start.

All team members took part in building the model

shaping use cases along with class layouts and data flow visuals. One helped define what the system should do, while another focused on how it should perform behind the scenes. Some drafted screen designs; others gave feedback during shared check-ins - keeping everyone aligned around one clear picture of the end product.

7.3 Communication Methods

To keep things running smoothly, we stuck to a WhatsApp chat where everyone dropped messages every day, shared quick check-ins, or made choices on the fly. That setup gave us room to adapt, letting us sync up anytime, not just during planned meetups. On top of that, we got together now and then - sometimes in person, sometimes through video chats - to go over how far we'd come, spot any issues, then figure out what to tackle next as a crew.

We used Google Docs along with Gmail for sharing files and working together. With these apps, it was simple to write reports side by side, tweak visuals, and pass around suggestions, so nobody missed updates on our latest drafts.

Even though we skipped official project tools or code tracking, using shared drives plus regular updates made sure all stayed on the same page. When big shifts in design or focus popped up during threat checks, we talked them through as a group prior to making moves. That way of working kept folks focused, in sync, because they had real say whenever choices mattered.

8. Risk Management Plan

Software projects come with risks - ours had even more because it's packed with security tools. Because of that, we put together a strategy to spot major threats, judge how likely they are and how bad things could get, then figure out ways to reduce or deal with each one.

Table1: Summary of project risks, their likelihood, effects, and the mitigation strategies applied.

Risk	Likelihood	Effect	Mitigation Strategy
Scope expanding unexpectedly	Medium	Big - might slow things down or result in a half-finished project	We applied the SQUARE approach to set solid requirements from the start - new features needed sign-off before moving forward. Security upgrades had fixed timelines, tied strictly to risks outlined in the threat analysis.
Technology Uncertainty	Medium	Moderate - could slow progress or reduce protection	We set aside time at the start for testing ideas plus digging into research, checked guides along with advice from experienced folks, so we could focus on clear, easy-to-follow tools that cut down guesswork.
Integration Issues	Medium	Moderate - might lead to redoing work or slow down	Parts got added step by step, while small checks made sure

		tech progress	encoding and decoding worked right - just in case, we held onto backup plans, shifting from full-path to channel-based protection for a short time if things went sideways.
Performance Overhead	Low	Fair - sluggish feedback might mess with how easy it is to use, especially since this thing's just a test version.	We picked fast algorithms such as AES-NI, carried out speed checks, while setting real-world caps - like max file size when encrypting. A few intense tasks got built to work in the background so the app stays responsive.
Security Design Flaw	Medium	Big - one weak spot might wreck everything the project stands for	We used STRIDE threat modeling, while peer code checks focused on security - alongside internal 'red team' drills. OWASP's top risks got a look; when issues couldn't be fixed right away, they were logged for later action.
Team members might be missing	Low	Big - few people working together can't cover much when someone's missing	We tried out job swaps, kept through notes that everyone could check, and also made sure to warn the teacher ahead of time

			whenever timing needed changes
--	--	--	--------------------------------

Every risk got a rough check. The biggest problem is the scope slowly growing - everyone wanted to toss in one extra safety fix. So we held tight to the first plan's list and goals. Stuff beyond that went into a maybe-later pile, not today's work.

Security gaps stayed a constant concern during building phases. To reduce them, we stuck to safe coding rules - using methods such as SQUARE and STRIDE, along with regular team checks on the code. Once each cycle wrapped up, we went over the risks again, tweaking the list based on what we'd learned. Take the first working model: once it was done, tech unknowns felt less scary since we knew the tools better; however, combining parts got trickier as things grew more involved. So, we shifted our approach to match.

If a problem popped up even after we took steps - like when encryption made things run too slow - we weren't caught off guard because fixes were lined up. Then, the end version might switch to a lighter method or just encrypt key parts, given how tight the schedule was, plus jot down upgrades for later. By staying alert and ready to adapt, we kept moving forward and hit core safety goals without chaotic scrambles at the end.

9. Functional and Non-Functional Requirements

In SecureFocus, a set of criteria was developed and sub-divided into functional and non-functional dimensions. Every need meets the SMART requirements—Specific, Measurable, Achievable, Relevant and Traceable.

9.1 Functional Requirements

FR1: User Account Management – It will allow new users to register in the system by providing a unique username and strong password, which will be confirmed with an

email message. Returning users must sign in with a username and password; multifactor authentication is available for added security.

FR2: Project Creation and Management – Authenticated users can create new project areas, assign titles and descriptions, and manage team membership. No, members can quit but project owners have the ability to add or remove people.

FR3: Task Management (Kanban Board) – Users are responsible for creating and managing tasks with properties like title, description, due date, and status inside each project. A Kanban-style interface that allows for drag-and-drop changes will be used to graphically manage tasks.

FR4: File Repository and Sharing – Project files must be uploaded, accessed, and managed by users. File owners have the ability to modify access rights, and uploaded files are connected to their corresponding projects.

FR5: Real-Time Communication – Team conversation with text, links, and optional attachments will be supported by an integrated project chat. The conversations will be permanently recorded in the chat.

FR6: Notifications System – Through the UI or email, users will receive reminders for important actions, such as work assignments, impending deadlines, or being referenced in communications.

FR7: Role-Based Access Control – There will be two roles: Member and Project Owner. Members can only change their own assignments and contributions; owners have administrative rights.

FR8: System Audit Logging – A comprehensive audit record that tracks important operations (such as login attempts, file access, and permission adjustments) will be available to administrative users.

FR9: Responsive Interface – In order to ensure usability on desktop, tablet, and mobile displays, the online user interface must smoothly adjust to different devices and browsers.

Every functional requirement is made to be testable (e.g., validating that a user can create and execute a task as defined in FR3).

9.2 Non-Functional Requirements

NFR1: Performance – Under typical circumstances, the system should provide responsive interactions, with essential tasks (such loading projects and sending messages) taking about three seconds to complete. At least 50 concurrent users should be supported without noticeably degrading performance.

NFR2: Usability – New users will be able to do the required tasks without formal teaching because to the interface's intuitiveness. Language and feedback signals must be clear and consistent. A usability test with at least ten individuals will be used to gauge usability; 80% of them should be able to effectively execute important activities without help, with an error rate of less than 10%.

NFR3: Reliability and Availability – The system must function dependably with minimal destruction of data or downtime. In the event of a system breakdown, backups will guarantee data recovery. Reliability will be assessed by maintaining 99% uptime every month, and backup recovery will be certified by executing quarterly restoration tests achieving full data recovery within 30 minutes

NFR4: Maintainability – The front-end, back-end, and database layers will be separated by a modular design used in the solution. Future maintenance and improvements will be supported by code documentation and uniform standards. Maintainability will be assessed by making sure that code documentation coverage reaches at least 80% and that every module-level change needs an average of 4 hours of development work.

NFR5: Portability – For wide compatibility, the system must be deployable on every popular operating system, with a focus on open-source, cross-platform technology. By successfully deploying and operating the system in at least three environments (Windows, Linux, and macOS) without any significant functional problems, portability will be confirmed.

NFR6: Security – All data must be secured while it's in transit and at rest (HTTPS/TLS). The system will use MFA, enforce strong passwords, and guard against common vulnerabilities (XSS, CSRF, SQL injection). Accountability and incident analysis will be aided by security records. Quarterly penetration testing that reveal no high-severity vulnerabilities, password strength enforcement that mandates a minimum of 12 characters, and MFA success rates that above 95% will be used to gauge security.

NFR7: Compliance – The system's architecture takes into account data protection requirements like GDPR, including procedures for data erasure upon user request, even if it is mostly academic. Compliance will be verified by confirming that data deletion requests are completely executed within 30 days and by performing yearly compliance audits to ensure procedures match policy requirement

Every non-functional need has quantifiable or verifiable standards that guarantee system performance, security, and quality are clearly fulfilled.

10. Security Requirements (Using the SQUARE Methodology)

To figure out what **SecureFocus** needed for safety, we used the SQUARE method. Because it's organized, this way let us go step by step - from grasping concerns straight through to building solid, checkable rules.

Here's what we did next, along with how things turned out at every point.

Stage 1: Agree on Definitions

Right from the start, we checked that everyone on the team got what words like secrecy, accuracy, verification, access rights, and responsibility really meant. While doing this, it helped us line up our thoughts so no one was guessing what others meant when talking about safety stuff.

This move mattered because it helped people see eye to eye on safety targets, so mix-ups wouldn't pop up down the line while talking about threats or safeguards.

Stage 2: Identify Security Goals

Once we talked through what the project should achieve, our team settled on these key security aims for SecureFocus

- **G1:** Keep user info safe from intruders, while shielding project details using strong barriers.
- **G2:** Keep data safe by blocking changes to tasks, unless permission is granted; stop file edits without approval; messages stay locked if access isn't authorized. Check who users are before letting them in - make sure first, then allow entry.
- **G4:** Keep track by using logs along with oversight.
- **G5:** Keep messages protected while also making sure files are managed without risk on the system.

These targets line up with what the project's really about - giving small groups a safe way to work together.

Stage 3: Develop Artifacts

This time round, we collected a bunch of helpful resources to back up how we check what's needed - like examples, notes, past cases, rough drafts, reference guides, real-life scenarios, quick summaries, and similar stuff

Models of the system display user actions during login, handling tasks, also sending messages.

Data flow diagrams (DFDs), used to track where info goes from client to server, then over to database.

Threat lists pointing out potential dangers such as unapproved access, info spills, or code injections.

Those old items showed us spots where the system could break, so we know which parts need stronger shields.

Stage 4: Perform Risk Assessment

We identified and prioritized potential security risks:

Table2: List of identified security risks with their impact, likelihood, and priority levels

Risk ID	Description	Impact	Likelihood	Priority
R1	Unauthorized access to user accounts	High	Medium	High
R2	Sensitive data exposure (e.g., unencrypted files)	High	Medium	High
R3	SQL injection or XSS attacks on web inputs	High	Medium	High
R4	Message or file tampering	Medium	Medium	Medium
R5	Weak password or credential reuse	Medium	High	High
R6	Lack of audit logging or accountability	Medium	Medium	Medium

From this, we decided to focus most effort on authentication, encryption, and secure coding practices.

Stage 5: Select Elicitation Techniques

We picked group talks to collect real needs, mixed in story-driven brainstorming sessions instead, then added a list of typical web risks - pulled straight from OWASP's Top 10 - as a backup layer.

This made linking potential threats to proper safeguards simpler for SecureFocus - using practical measures that fit each case naturally.

Stage 6: Elicit Security Requirements

Pulling from what we looked at before, so we gathered clear security needs - numbered SR1 through SR10 - that match up with our aims as well as possible threats.

We checked these against the non-functional needs too, using that chance to dodge repeats or clashes.

Stage 7: Categorize Requirements

Every need got sorted into at least one key security trait - like Confidentiality, Integrity, Availability - not just those, but also Authentication, plus Authorization, along with Accountability.

Stage 8: Prioritize Requirements

Priorities got set by how risky something was, whether it could actually be done, or what effect it'd have on the system. Big issues tackle clear dangers - say, someone breaking in or info getting leaked - whereas mid-level ones aim at boosting performance or staying within rules.

Stage 9: Inspect and Validate

Lastly, we went over every security need we found with our mentor - then matched each one against what the project aimed to achieve and its limits. One by one, they were checked to make sure they made sense, could be tracked back, and actually worked within how far our test version reached.

Table3: Security requirements categorized by priority and traceability.

ID	Security Requirement	Category	Priority	Traceability
SR1	The system shall enforce strong password policies (minimum length, complexity, and no reuse).	Authentication	High	FR1
SR2	The system shall support multi-factor authentication (MFA) for login.	Authentication	High	FR1
SR3	All data in transit shall be protected using HTTPS/TLS encryption .	Confidentiality	High	NFR6
SR4	All stored data (e.g., files, messages) shall be encrypted at rest on the server.	Confidentiality	High	NFR6
SR5	The system shall log all critical security events , including logins, file downloads, and permission changes.	Accountability	Medium	FR8
SR6	The system shall restrict file and task access based on user roles (Owner/Member).	Authorization	High	FR7
SR7	The system shall sanitize all user inputs to prevent XSS, CSRF, and SQL injection attacks.	Integrity	High	FR3, NFR6
SR8	The system shall notify users of suspicious login attempts or access from new devices.	Accountability	Medium	FR6

SR9	The system shall provide regular backups to allow data recovery in case of system failure.	Availability	Medium	NFR3
SR10	The system shall limit login attempts to mitigate brute-force attacks.	Authentication	High	FR1

11. System and Design Modeling

11.1 High-Level System Architecture

SecureFocus runs on a layered setup where each part handles different jobs, keeping things organized and safe. It's built with three main sections: what users see, the engine behind it, and the place where data lives. The visible side lets people log in, manage tasks, send files, and message teammates using encryption. Whenever someone types something or clicks around, those actions get turned into secure messages headed to the backend through HTTPS. The backend runs the show - handling logins, permissions, assigning jobs, managing chat flow, and more - all while talking to a protected database holding user profiles, job history, uploads, plus message trails locked up tight.

Files, login codes, along conversations. They're scrambled on arrival by built-in encryption tools, only unlocked when someone legit needs them. Data moving between systems rides through secure tunnels, whereas sneaky attacks like code injections or script hacks get blocked early by a smart filter standing watch in the middle layer. Built this way, it grows smoothly without cutting corners on keeping info safe from start to finish.

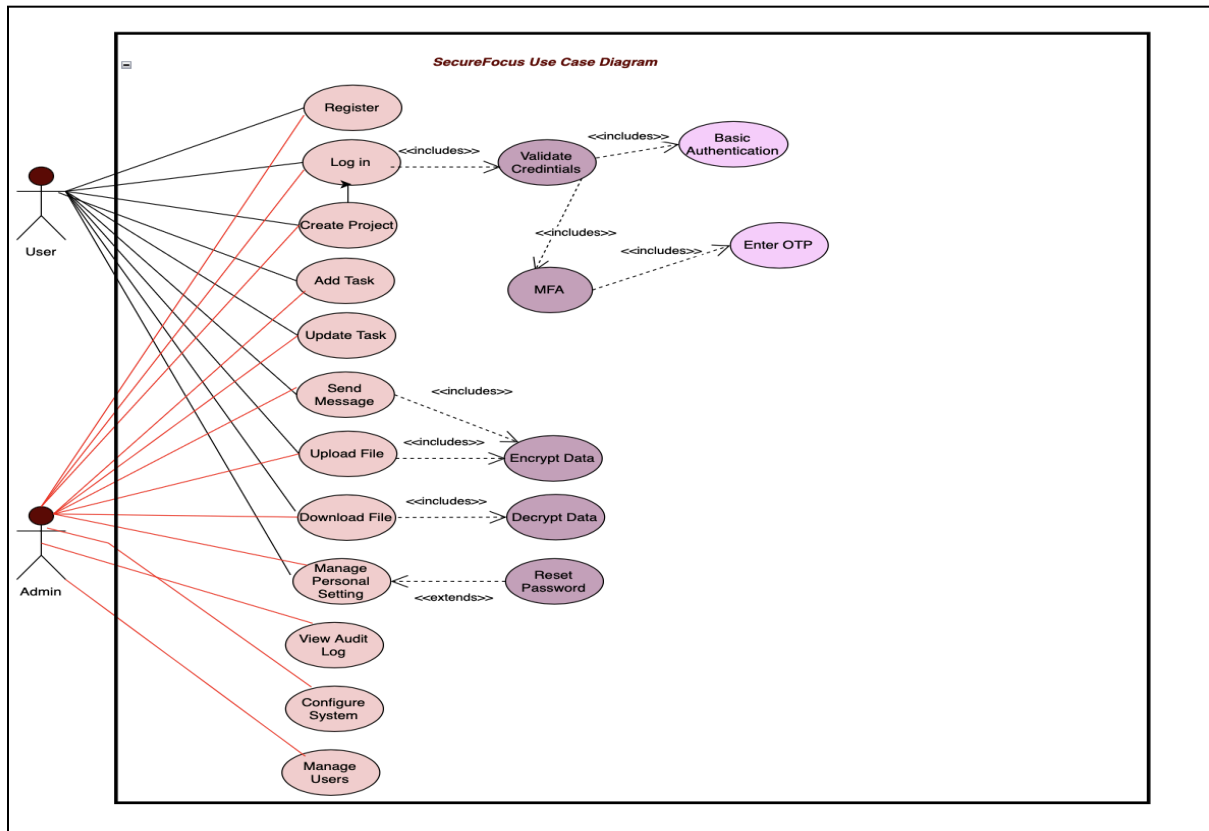
11.2 Use Case Diagram

The SecureFocus Use Case diagram displays the system interactions with two main user types: standard user and an admin. Standard users can set up projects, login, register, or update tasks

whenever they want. They can also share encrypted messages, exchange files, and manage their own profile settings. Security is a great deal here, once someone sends a message or uploads a document online, encryption occurs automatically. Once files are downloaded, they get decrypted immediately, this keeps data private and secured. Logging in includes multi-factor authentication (MFA), where you enter your credentials first, followed by a one-time password (OTP).

Admins do everything regular users can, but with added authority. They're able to manage users, configure system settings, and review logs showing who did what and how the system runs. Because of this, security stays strong while daily functions run without disruptions. The diagram connects common actions like encryption, decryption, and authentication, through "include" and "extend" relationships. In short, it maps out how users engage with SecureFocus, highlighting secure collaboration and controlled administration.

Figure1: Use case diagram showing user and admin interactions with the system.



11.3 Class Diagram

The SecureFocus Class diagram shows what makes up the system. Its main classes, their attributes, methods, and how they connect with one another. In the middle you'll find the class Standard User. It includes attributes like userID, name, email, passwordHash, and role, and allows actions like log in or out, update tasks, upload and download files, send messages, and also handle account settings. The Administrator gets all these functions but goes beyond, enabling checking activity logs, handling user accounts, and changing system options across the platform.

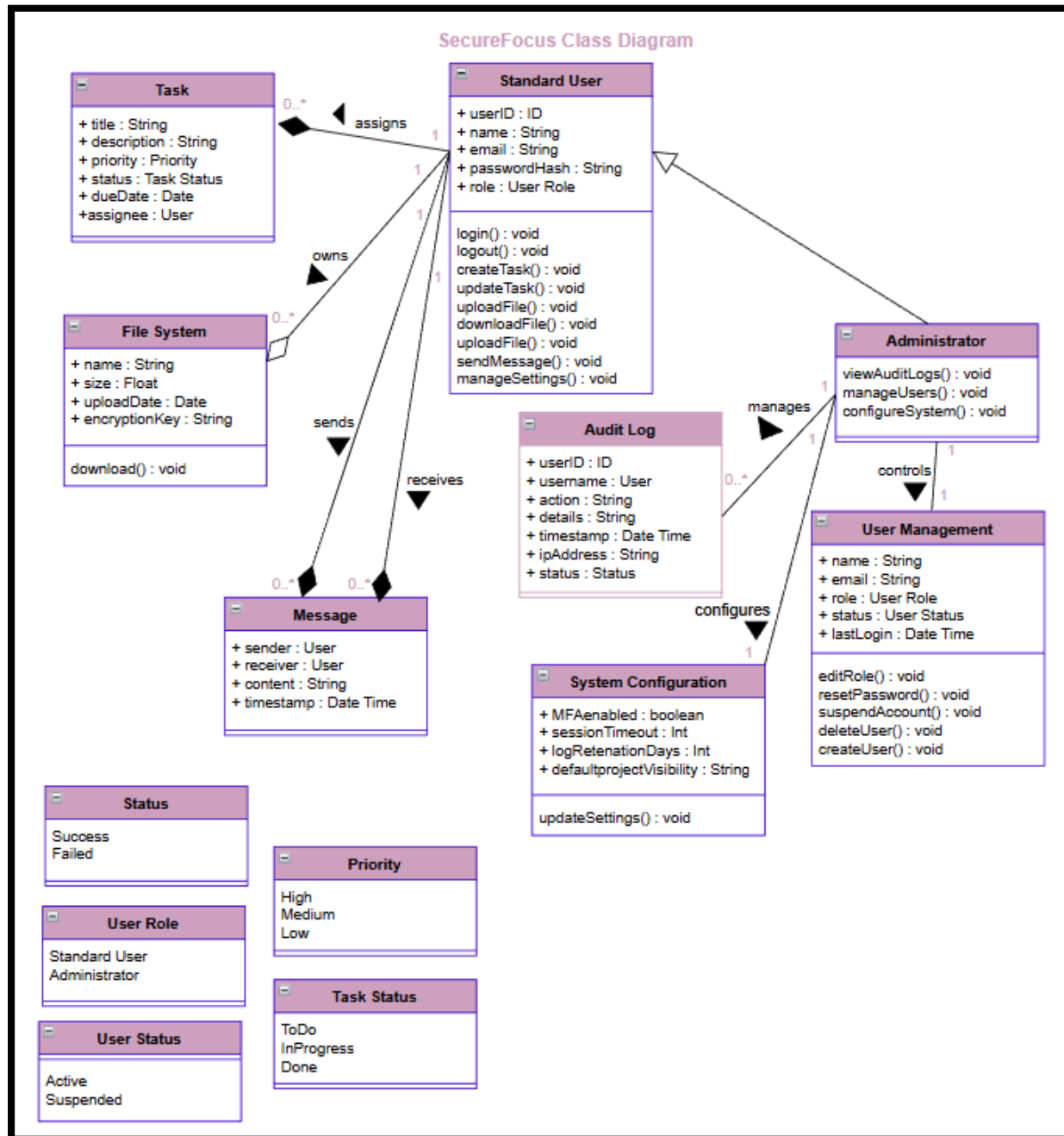
The Task class defines tasks through title, details priority, status, due date, and assignee. Tasks are connected to Standard User through an aggregation relationship. The Message class handles

encrypted communication, including sending and receiving. Next comes the File System class, where it is in charge of managing encrypted downloads and uploads. Lastly, the Audit Log tracks any security-related event through timestamps, IPs, results and all about it.

The System Configuration class handles how operations run, controlling MFA enablement, setting session timeouts, and also audit log retention periods. Admins take care of User Management, which includes adjusting roles, resetting passwords, suspending accounts, deletion of accounts, and creating new users in the system.

Several fixed lists like User Role, User Status, Task Status, Priority, and overall Status, help keep values uniform throughout the app. With clear links showing aggregation and composition relationships, the model shows exactly where everything belongs while clarifying how it all ties together, offering a clean view of how SecureFocus is built.

Figure2: Class diagram illustrating the main system entities, their attributes, methods, and relationships within the SecureFocus system.



11.4 Data Flow Diagram (DFD):

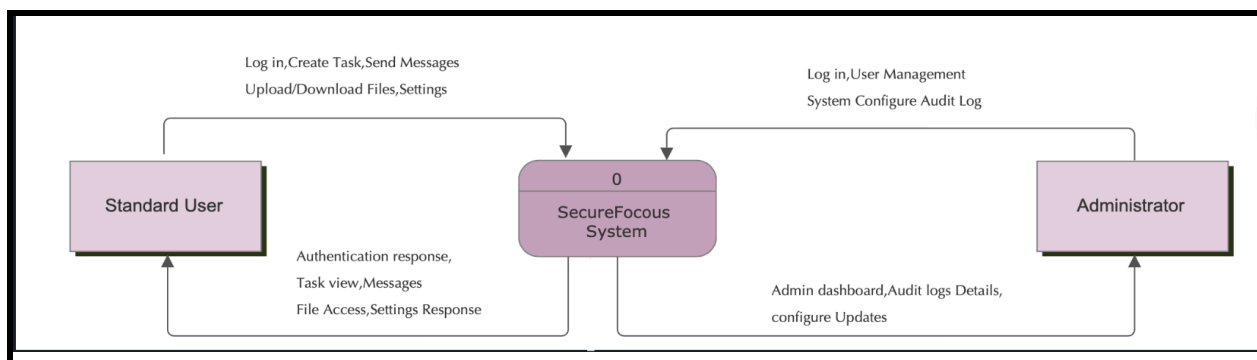
Level 0

The Level 0 DFD for SecureFocus shows a top-level view of how the system works, highlighting key processes, data stores, and external entities. It traces how data flows between the two primary users, standard user and admin, with the core SecureFocus system in the middle.

A standard user logs into the system, creates and manages tasks, sends encrypted messages, uploads or downloads encrypted files, and adjusts personal settings. The platform sends back authentication outcomes, updated task layouts, chat responses, file access, and confirmation signals when settings get changed. These flows are supported by the system's data stores: D1: User Accounts, D2: Task Database, D3: Message Encrypted, D4: File Storage Encrypted.

Admins log in the same way, but have more control. They manage user accounts, configure settings, and check activity records. As a result, they get dashboard data, summaries, and configuration feedback. These actions rely on D5: Audit Logs and Settings. The SecureFocus setup runs as a single central process called Process 0, that manages movement between users and stored data. The diagram sets boundaries between user interactions and inner workings, forming a base for detailed DFDs that focus on individual processes later.

[Figure3: Level-0 DFD of SecureFocus showing user interactions and data flows.](#)



Level 1

The Level 1 DFD for SecureFocus goes further than Level 0, breaking down the system into six detailed processes. It tracks how data flows within, how regular users and admins engage with different modules, and highlights which data stores manage incoming requests.

Process 1.0: Authentication

It all starts right here. Users and admins login using an email and password, which get matched up with records in User Accounts (D2). If everything lines up, access is given based on role, then a success message pops up.

Process 2.0: Tasks

Once logged in, regular users are able to create, edit, or review their tasks. These actions link up with the Task Database (D5). Updates, completions, or new entries flow back as task status feedback to the user.

Process 3.0: Messages

This part takes care of encrypted communication. Each message, whether sent, seen, or retrieved, is stored safely in Message Storage (D3). Encryption here ensures privacy end-to-end.

Process 4.0: Files

All file uploads and downloads move through this process. Once uploaded, they're kept safe in Encrypted File Storage (D4), only accessible with proper verification.

Process 5.0: Admin Panel

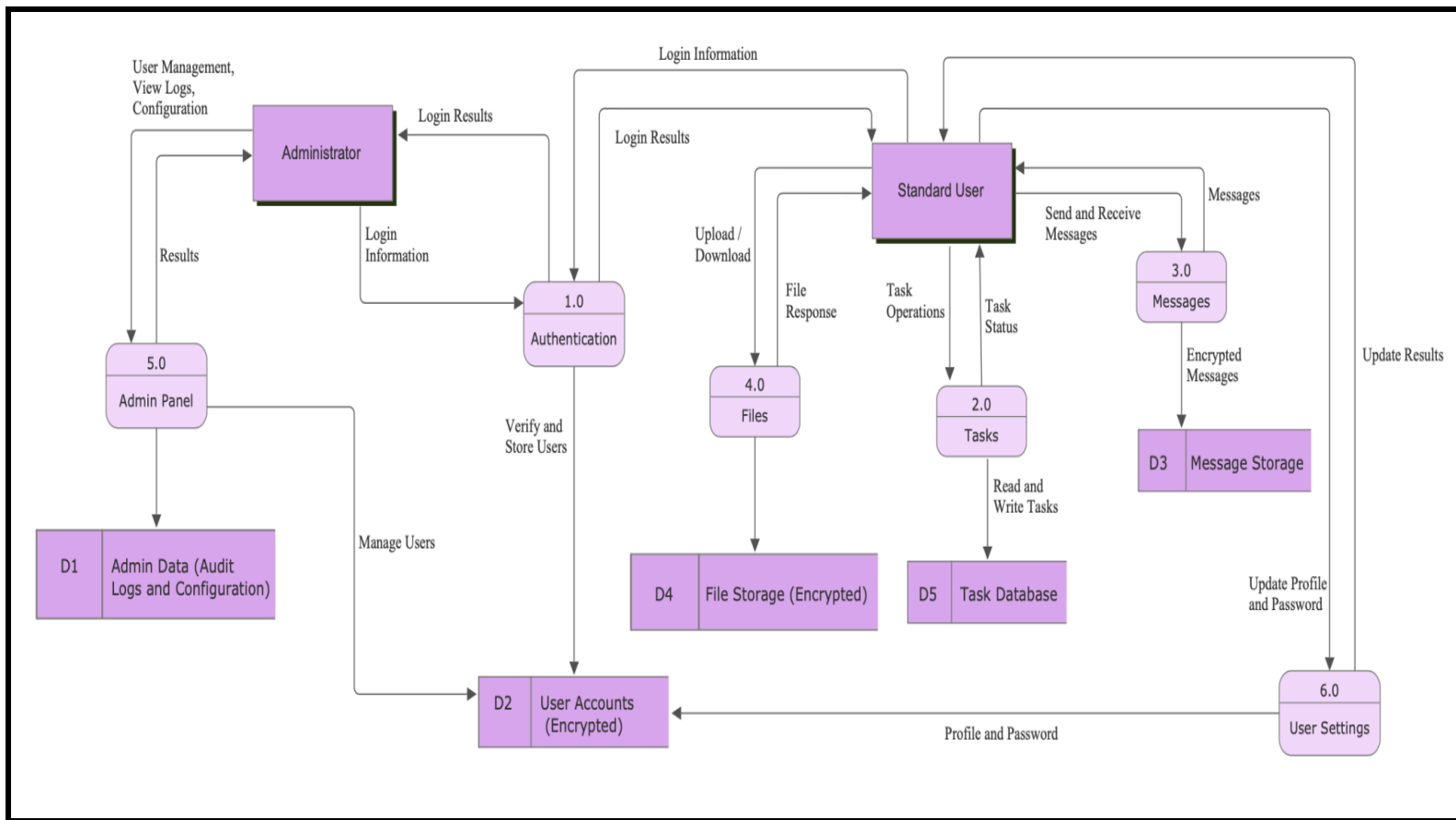
Only admins have access to this. They manage users, check system logs, and tweak configurations. This whole workflow connects with Admin Data (D1), containing audit logs and system configurations. Each admin action pushes or retrieves relevant data from here.

Process 6.0: User Settings

Here users, usually admins, can update personal information, reset passwords, and adjust preferences. Changes get sync securely with User Accounts (D2).

Overall, the diagram shows a structured and secure setup where each major task requires authentication. Each module operates with a clear route, uses separate encrypted storage, while enabling full traceability through audit logging. The design stacks levels, is strict to rules, and transparent.

Figure4: Level-1 DFD detailing the internal processes and data interactions within the SecureFocus system.

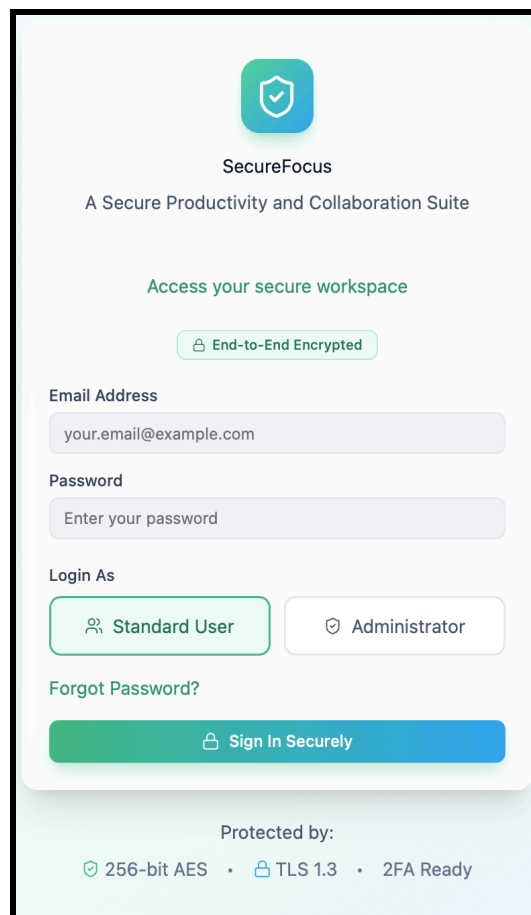


11.5 Interface Design: Figma

(<https://clever-cake-97784513.figma.site>)

A safe login page for SecureFocus - includes fields for email and password. It offers two ways to login: one for regular users, another for admins. It shows key protections like - end-to-end encryption, 256-bit AES encryption, TLS 1.3, and 2FA readiness.

[Figure5: SecureFocus login interface showing authentication fields, role options, and security indicators.](#)

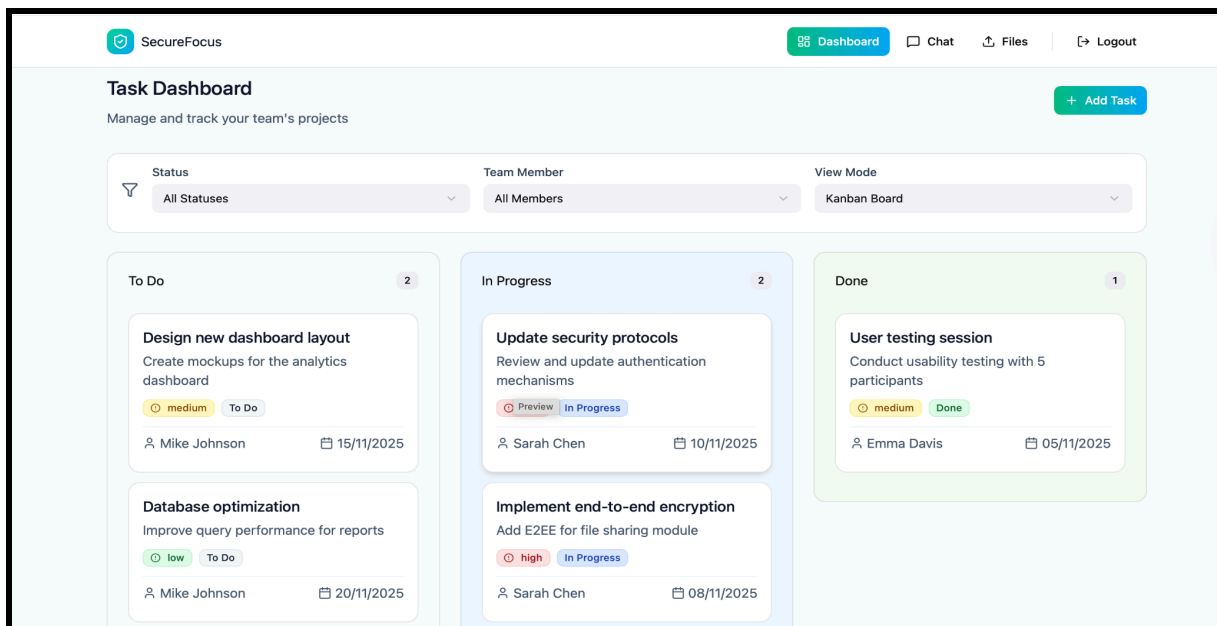


The image is a mockup of a login interface for 'SecureFocus'. At the top, there is a blue shield icon with a white checkmark. Below it, the text 'SecureFocus' is centered, followed by 'A Secure Productivity and Collaboration Suite'. A green link 'Access your secure workspace' is centered below that. A green pill-shaped button with a lock icon and the text 'End-to-End Encrypted' is centered. Below this, there are two input fields: 'Email Address' with the placeholder 'your.email@example.com' and 'Password' with the placeholder 'Enter your password'. Under the password field, there are two buttons: 'Standard User' (with a person icon) and 'Administrator' (with a key icon). Below these is a link 'Forgot Password?'. A large blue button with a lock icon and the text 'Sign In Securely' is centered. At the bottom, the text 'Protected by:' is centered, followed by three items: '256-bit AES' (with a shield icon), 'TLS 1.3' (with a lock icon), and '2FA Ready' (with a checkmark icon).

Standard User View:

This is the main workspace where standard users work. Tasks get split into three groups - To Do, In Progress, or Done. Filtering options let you sort by status, member, depending on what's needed. Switching from Kanban layout to a simple list is possible too. Navigation buttons include Chat, Files, and a Logout option.

Figure6: Standard user dashboard displaying tasks organized by status and project workflow.



A little window pops up to add new tasks. You type a name, description, pick a priority level, assign it to a member, then set its due date. It ends with the “Create Task” button to save it.

Figure7: Task creation window for adding a new task with details, priority, assignee, and due date.

The image shows a 'Task Dashboard' interface with a 'Create New Task' modal window open. The modal is titled 'Create New Task' and has a close button (X) in the top right corner. Below the title is the instruction 'Add a new task to your project board'. The form contains the following fields:

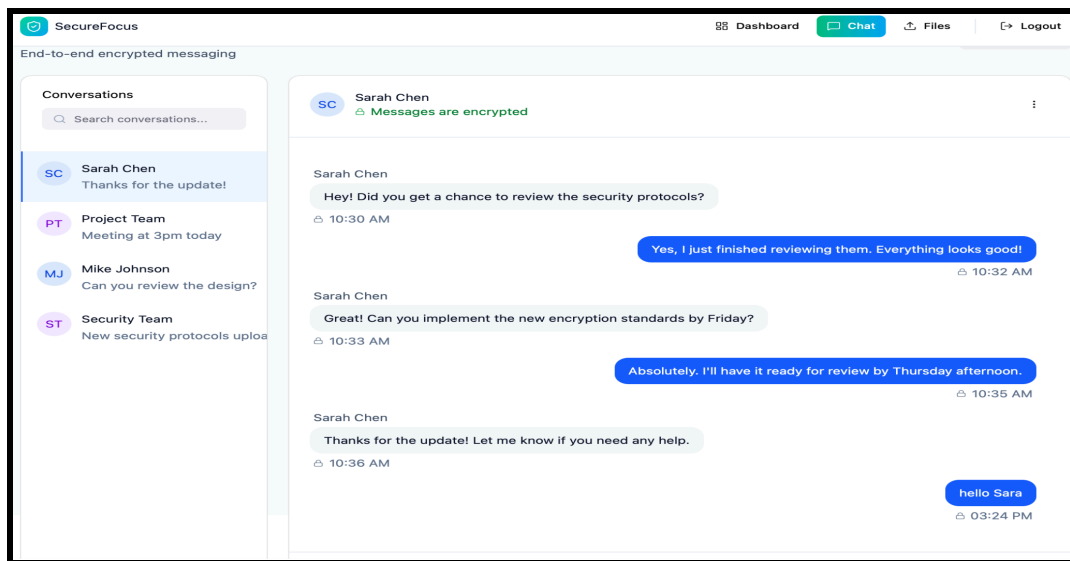
- Task Title:** A text input field with the value 'prepare for testing seesion'.
- Description:** A text input field with the value 'prepare the environment'.
- Priority:** A dropdown menu with the value 'High'.
- Assignee:** A text input field with the value 'Sarah'.
- Due Date:** A date input field with the value '06/11/2025'.

At the bottom of the modal are two buttons: 'Cancel' and 'Create Task'.

The background dashboard shows a 'Status' filter set to 'All Statuses', a 'To Do' section with a task 'Design new dashboard layout' (medium priority, due 15/11/2025), and a 'Database optimization' task (medium priority, due 05/11/2025).

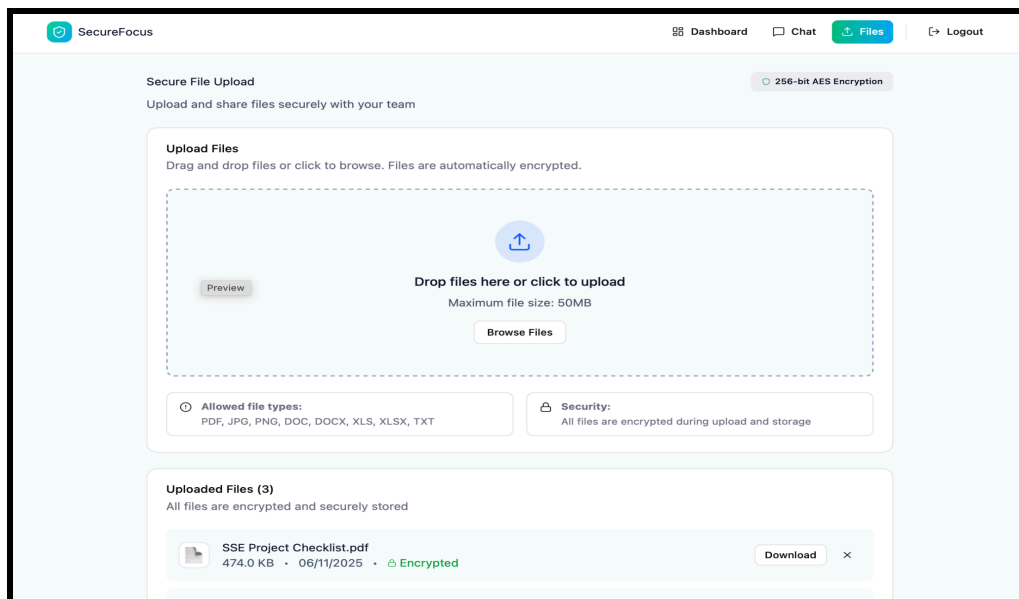
A secure messaging space where teammates can chat safely. One side displays ongoing chats, while the opposite holds live messages. Members pick contacts from the sidebar before typing anything. Messages pop up instantly when sent during active windows.

Figure8: SecureFocus messaging interface showing encrypted conversations and chat threads.



This is a secure page for sharing documents. Users can drag and drop files here. It displays details on encryption, file size and type limits, and shows uploaded files with download options.

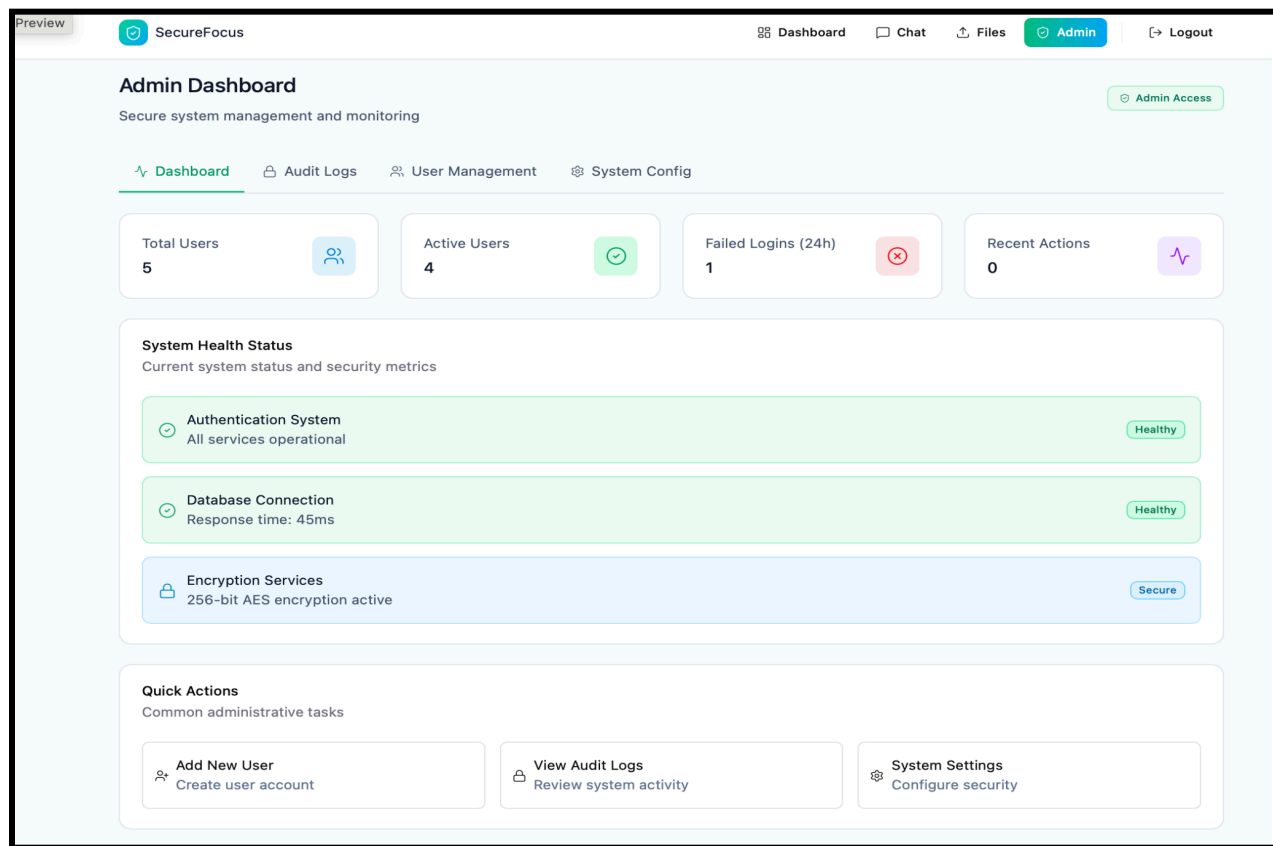
Figure9: Secure file upload interface showing encrypted file uploads and stored files.



Administrator View:

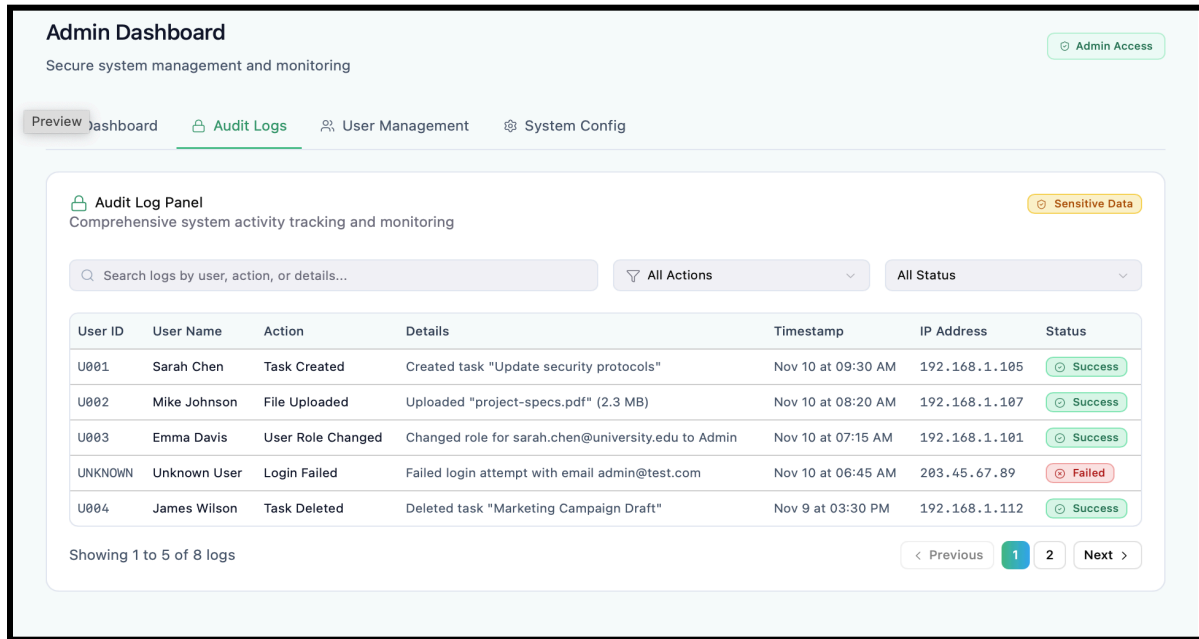
Here is the admin's main dashboard. It displays data such as user count, active users, login failures, and recent activities. From here, admins can perform quick actions while monitoring system health indicators.

Figure10: Admin dashboard displaying system health, user statistics, and key administrative actions.



A clear log area that tracks system actions. Every entry lists the user ID, action, timestamp, IP address, and if it succeeded or failed. There are also filter and search options available for specific record search.

Figure11: Admin audit log panel showing detailed system activity records with filtering options.

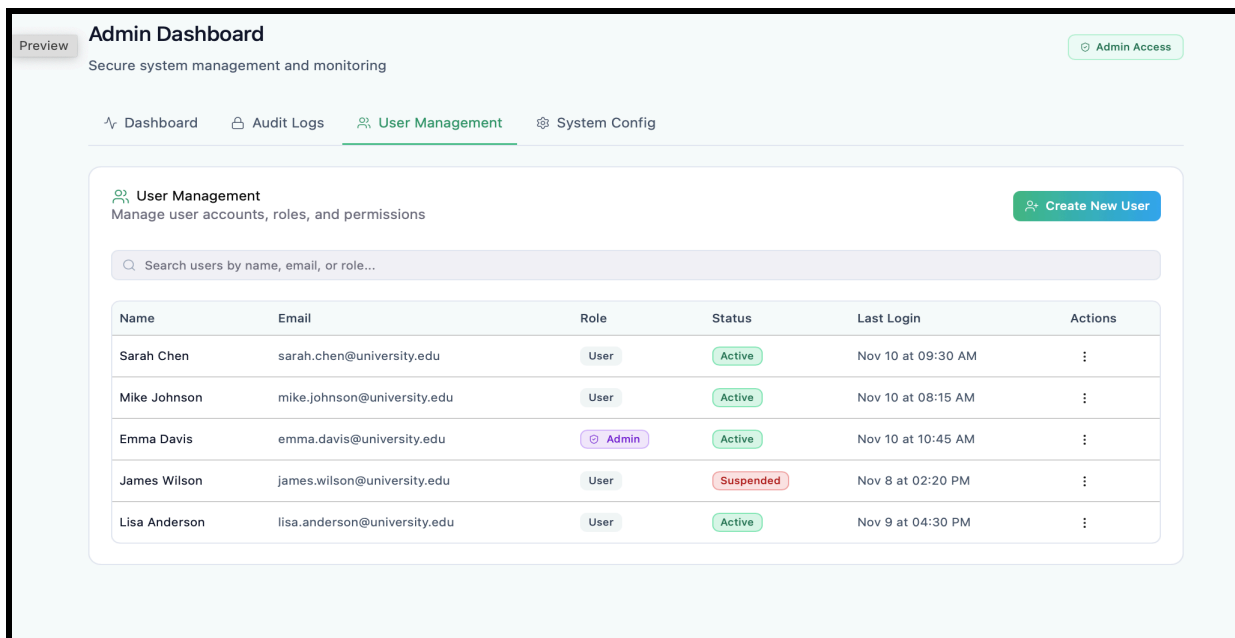


The screenshot shows the 'Admin Dashboard' with the 'Audit Logs' tab selected. The dashboard title is 'Admin Dashboard' with the subtitle 'Secure system management and monitoring'. A navigation bar includes 'Preview', 'Dashboard', 'Audit Logs' (active), 'User Management', and 'System Config'. A 'Sensitive Data' warning is present. The 'Audit Log Panel' section has a search bar and filters for 'All Actions' and 'All Status'. It displays a table of system activity logs with columns: User ID, User Name, Action, Details, Timestamp, IP Address, and Status. The table shows five logs, with the first four being successful and the last one failed. A pagination bar at the bottom indicates 'Showing 1 to 5 of 8 logs'.

User ID	User Name	Action	Details	Timestamp	IP Address	Status
U001	Sarah Chen	Task Created	Created task "Update security protocols"	Nov 10 at 09:30 AM	192.168.1.105	Success
U002	Mike Johnson	File Uploaded	Uploaded "project-specs.pdf" (2.3 MB)	Nov 10 at 08:20 AM	192.168.1.107	Success
U003	Emma Davis	User Role Changed	Changed role for sarah.chen@university.edu to Admin	Nov 10 at 07:15 AM	192.168.1.101	Success
UNKNOWN	Unknown User	Login Failed	Failed login attempt with email admin@test.com	Nov 10 at 06:45 AM	203.45.67.89	Failed
U004	James Wilson	Task Deleted	Deleted task "Marketing Campaign Draft"	Nov 9 at 03:30 PM	192.168.1.112	Success

A section to manage all user accounts. It lists users' name, email, role, account status, and their last login. Admins can change roles, reset passwords, suspend, or delete accounts.

Figure12: Admin user management panel showing user accounts, roles, statuses, and login activity.

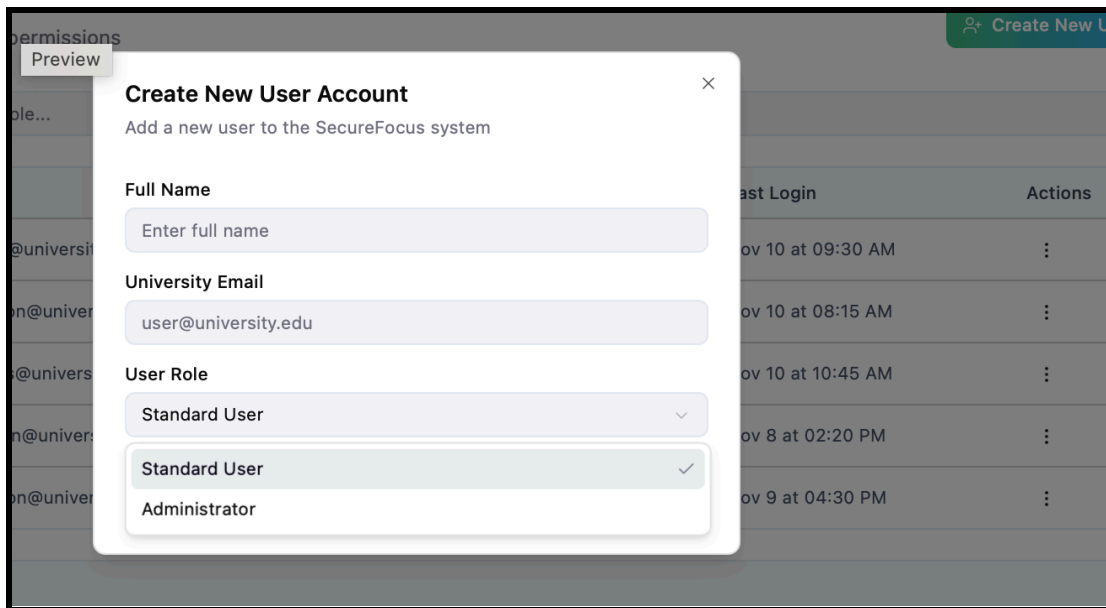


The screenshot shows the 'Admin Dashboard' with the 'User Management' tab selected. The dashboard title is 'Admin Dashboard' with the subtitle 'Secure system management and monitoring'. A navigation bar includes 'Preview', 'Dashboard', 'Audit Logs', 'User Management' (active), and 'System Config'. A 'Create New User' button is present. The 'User Management' section has a search bar and displays a table of user accounts with columns: Name, Email, Role, Status, Last Login, and Actions. The table lists five users: Sarah Chen (User, Active), Mike Johnson (User, Active), Emma Davis (Admin, Active), James Wilson (User, Suspended), and Lisa Anderson (User, Active). Each user has a vertical ellipsis icon in the Actions column.

Name	Email	Role	Status	Last Login	Actions
Sarah Chen	sarah.chen@university.edu	User	Active	Nov 10 at 09:30 AM	⋮
Mike Johnson	mike.johnson@university.edu	User	Active	Nov 10 at 08:15 AM	⋮
Emma Davis	emma.davis@university.edu	Admin	Active	Nov 10 at 10:45 AM	⋮
James Wilson	james.wilson@university.edu	User	Suspended	Nov 8 at 02:20 PM	⋮
Lisa Anderson	lisa.anderson@university.edu	User	Active	Nov 9 at 04:30 PM	⋮

A quick way to add new users. Admins enter the full name, email , then pick a role - regular member or admin.

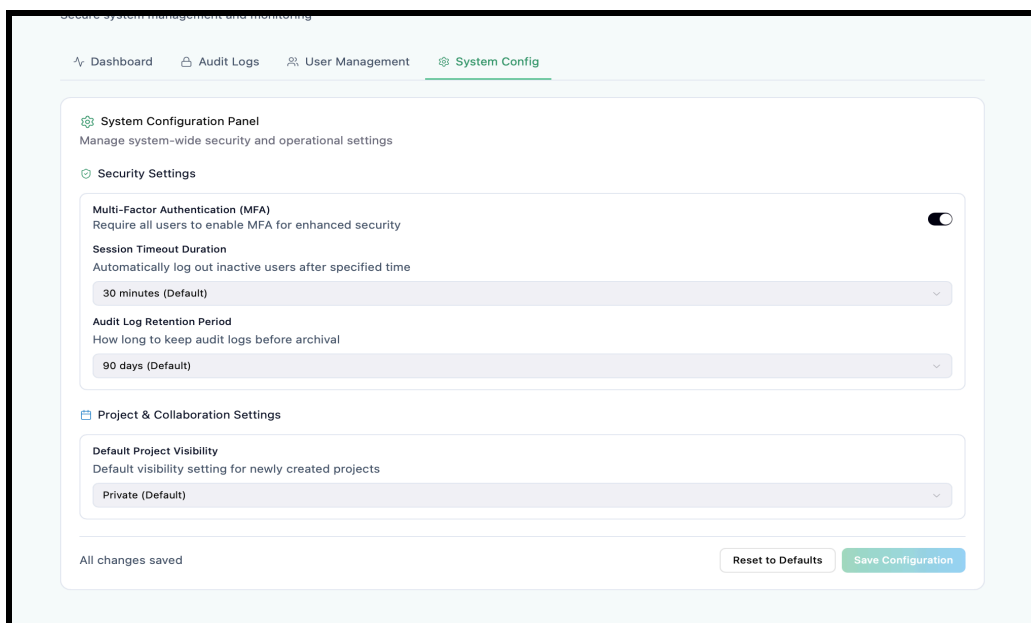
[Figure13: Admin form for creating a new user account with name, email, and role selection.](#)



The screenshot shows a modal window titled "Create New User Account" with a close button (X) in the top right corner. Below the title is the instruction "Add a new user to the SecureFocus system". The form contains three fields: "Full Name" with a placeholder "Enter full name", "University Email" with the placeholder "user@university.edu", and "User Role" which is a dropdown menu. The dropdown menu is open, showing three options: "Standard User" (selected with a checkmark), "Standard User" (with a downward arrow), and "Administrator". In the background, a table with columns "Last Login" and "Actions" is partially visible, showing login times like "ov 10 at 09:30 AM".

A settings section for admins. You can enable MFA, set session timeout limits, managing audit log retention, and also adjusting default project visibility. Comes with options to either reset or save changes.

[Figure14: System configuration panel with security and project settings.](#)



The screenshot displays the "System Configuration Panel" within a web application. The panel has a title "System Configuration Panel" and a subtitle "Manage system-wide security and operational settings". It is divided into two main sections: "Security Settings" and "Project & Collaboration Settings". Under "Security Settings", there are three settings: "Multi-Factor Authentication (MFA)" with a toggle switch, "Session Timeout Duration" with a dropdown menu set to "30 minutes (Default)", and "Audit Log Retention Period" with a dropdown menu set to "90 days (Default)". Under "Project & Collaboration Settings", there is one setting: "Default Project Visibility" with a dropdown menu set to "Private (Default)". At the bottom of the panel, there is a status message "All changes saved" and two buttons: "Reset to Defaults" and "Save Configuration". The top of the interface shows a navigation bar with links to "Dashboard", "Audit Logs", "User Management", and "System Config".

11.6 Secure Architectural Principles

The architecture of SecureFocus is built following a holistic set of secure design principles. We practice least privilege by providing users and components with only necessary levels of access, both role-based in terms of applications and via infrastructural arrangements.

The system follows a defense-in-depth strategy by combining multiple safeguards such as HTTPS, storage encrypting, MFA, and audit logging to protect against threats even if one security measure is breached. We work with secure property settings, short-term sessions, and mandatory MFA. We practice secure failure modes by implementing appropriate operation blocking even when system failures occur. Our design follows separation of concerns by designing application subsystems such as authentication and task management in a manner separated for easier evaluation.

We choose simplicity over complex designs to avoid mistakes by using tested approaches for encrypting components and simple role models. We practice complete mediation by implementing security screenings on every request for access via backend frameworks. We demonstrate accountable practices by following append-only logging due to any user activity for traceability and non-repudiation purposes.

Finally, we have considered secure design patterns by referencing STRIDE categories throughout design levels for pre-emptive designs to protect against threats including spoofing, tampering, repudiation, information disclosure, denial of service, and privilege escalation.

12. Threat Modeling and Security Analysis

To keep SecureFocus strong against typical threats, we ran a risk check using the STRIDE method - breaking down each danger step by step so nothing slips through unnoticed.

This made it easier to spot risks step by step, see how bad they could get, while also working out solid ways to reduce the harm.

The process began with checking key parts of the system - like logging in, handling tasks, sending files, or chatting - while spotting where different STRIDE risks might show up.

12.1 Overview of STRIDE Categories

Table4: STRIDE threat categories with their meanings and the security objectives they violate.

STRIDE Category	Meaning	Typical Objective Violated
S – Spoofing	Impersonating another user or system entity	Authentication
T – Tampering	Unauthorized modification of data or code	Integrity
R – Repudiation	Denying having performed an action	Accountability
I – Information Disclosure	Unauthorized access to information	Confidentiality
D – Denial of Service (DoS)	Making a system unavailable to legitimate users	Availability
E – Elevation of Privilege	Gaining higher access rights than intended	Authorization

12.2 Threat Identification and Analysis

The table below lists the threats found in SecureFocus, along with where they come from, what problems they might cause, or how to reduce them.

Table5: Threat analysis table outlining STRIDE threats, impacts, and proposed mitigations.

Threat ID	Category (STRIDE)	Description / Threat Source	Potential Impact	Proposed Mitigation
T1	Spoofing	An attacker attempts to log in using stolen or guessed credentials.	Unauthorized access to user accounts and private projects.	Enforce strong password policy (SR1), enable multi-factor authentication (SR2), and apply login attempt limits (SR10).
T2	Tampering	Malicious users attempt to modify project data, tasks, or files directly through URL manipulation or injection attacks.	Data corruption or unauthorized task/file changes.	Implement input validation and sanitization (SR7), enforce role-based access control (SR6), and use database parameterization.

T3	Repudiation	A user performs an action (e.g., deleting a file) and later denies doing it.	Difficulty in tracking malicious or accidental actions; lack of accountability.	Maintain detailed audit logs (SR5) with timestamps and user IDs; restrict log modification to administrators only.
T4	Information Disclosure	Sensitive data (e.g., files, chat messages) is accessed by unauthorized users through weak permissions or unencrypted channels.	Confidential project information leaked to outsiders.	Encrypt data in transit (SR3) and at rest (SR4); enforce access permissions per project role (FR7).
T5	Denial of Service (DoS)	Attackers flood the server with excessive requests, causing system slowdown or crashes.	Reduced availability or complete service outage.	Apply rate limiting, use load balancing, and monitor for abnormal traffic patterns.
T6	Elevation of Privilege	A user exploits a software vulnerability to gain admin rights or unauthorized project ownership.	Full system compromise, data breaches, or admin control loss.	Apply secure coding standards, conduct privilege validation on every request, and review code for authorization flaws.

T7	Spoofing/Information Disclosure	Email verification or password reset links intercepted through phishing.	Account compromise or identity theft.	Use HTTPS for all links, expire tokens quickly, and notify users of password changes.
T8	Repudiation/Tampering	Users post harmful or fake messages in project chat and deny responsibility.	Miscommunication, reputation damage, and loss of trust among team members.	Preserve message history with immutable timestamps and user IDs; allow admins to moderate messages.
T9	Information Disclosure	Uploaded files remain on the server even after a project is deleted.	Leakage of sensitive project files.	Enforce secure file deletion routines and periodic storage audits.
T10	Denial of Service / Elevation of Privilege	A member tries to overload system resources or exploit API endpoints for unauthorized actions.	Service instability and privilege escalation.	Validate API requests, use authentication tokens with expiration, and apply server-level rate controls.

12.3 Threat Prioritization

Once we checked every risk based on how bad it could be and how likely, we focused our fixes where they mattered most

1. Auth checks stop fake access plus block higher-level takeovers (T1, T6).

2. Data security – Stop changes or leaks using coding plus checks (T2, T4, T9).
3. Keep things running smoothly by blocking attacks that slow down services or sneak in bad data (T5, T10).
4. Tracking who does what, plus making sure it's checkable later on (T3, T8).

This focus means SecureFocus tackles serious threats early - yet still keeps protection steady in every STRIDE area.

12.4 Summary

By using STRIDE to map threats, it became easier to spot weak spots in SecureFocus - also showing ways to block issues before they start during planning.

Many risks get handled well by current safety rules (SR1–SR10), like using two-step login along with cleaning user inputs, encrypting information, or keeping track via logs.

So, SecureFocus's setup today includes several protection levels that lower the chance of attacks - and also limit their effects - helping keep user teamwork safe and steady.

13. Evaluation and Discussion

While building SecureFocus, our team looked at how design decisions paired up with real-world results - this helped spot key compromises, hurdles, plus insights guiding the end product. We zeroed in on how well it blocked threats, whether people could use it without hassle, and if the tech actually worked given limited time and tools common in school-driven projects.

13.1 Trade-Offs

During development, one choice often meant sacrificing another - security sometimes took a hit when speed mattered more; meanwhile, ease of use shaped decisions just as much as protection did

Security vs. Usability:

Adding things like two-step verification along with rigid password demands made the system safer, though it made signing in a bit harder now and then. This compromise was fine by us since keeping logins tough to crack mattered way more when guarding personal info plus sensitive work files.

Speed or safety when handling info?

Storing all data encrypted while relying on HTTPS across connections made the server work harder. Still, protecting information took priority instead of speed - because with just a few users, responses stayed quick enough.

What's included versus when it's done:

Some upcoming tools - such as smarter admin panels along with auto-based compliance tracking - got delayed so we could roll out a solid, safe base system early. That choice let us keep the code clean while preventing hasty launches of tough security bits before full test runs.

13.2 Challenges Faced

Some issues came up while collecting needs, working on layout, also checking results:

Integrating Security from the Start:

Starting off with SQUARE plus STRIDE felt tricky at first - had to spot risks and fixes way before coding really began. Still, pushing through that upfront work kept big security gaps from popping up down the line.

Balancing Technical Complexity:

Some safety setups - like strong encryption or multi-step logins - were tough to build. Since our test setup needed specific tools, we checked several libraries, tried them out, then picked the ones that actually worked well together.

Data Handling and Permissions:

Setting up a system where users had specific roles - working smoothly no matter if they were handling tasks, files, or chats - meant rebuilding how info was stored. Even tiny errors in who could do what might let someone see too much or gain extra power, which dragged out the whole process.

Testing Security Features:

Testing real-life hacks - say, SQL injection or cross-site scripting - was tough without specific software. Instead of automated checks, we leaned on hand inspections along with team members double-checking each other's work to make sure safeguards actually did their job.

13.3 Lessons Learned

Working on SecureFocus taught us a lot - each step brought new lessons that shaped how we see security tools now

1. Security Must Be Continuous:

You can't just tack on safety stuff at the end. Instead, protection needs to weave through each phase - starting from planning and layout all the way through setup and upkeep.

2. Simplicity Improves Reliability:

We found out a simpler, modular setup cuts down on weaknesses while making fixes quicker - since complicated setups often cover up small safety issues.

3. Documentation and Traceability Are Critical:

Maintaining clear links from functional to non-functional and security needs kept our coverage complete while preventing oversights - so checking results and sharing updates became way easier.

4. User Experience Still Matters:

When a system's built to be safe, folks might skip safeguards just because things feel clunky or slow. We figured out that making it easy to use matters as much as keeping risks low.

5. Collaboration Enhances Security Awareness:

Chatting in groups or checking each other's work usually caught bugs a single coder could miss. Talking things through now and then made the whole system way more secure.

14. Future Work

While SecureFocus did hit its core goals - offering safe teamwork tools - some parts could still get upgrades down the line. Instead of staying put, later versions might boost speed, add useful features, or test how well protections hold up when things get messy in actual use.

14.1 System Implementation and Deployment

The latest build of SecureFocus runs like a rough demo.

Later efforts might turn this into a live web app running securely on cloud platforms like AWS, Azure, or Google's system.

This would include:

- Putting together CI/CD workflows so tests run automatically plus changes roll out smoothly.
- Setting up servers that grow with more users, while managing tough tasks through flexible design instead of fixed limits.
- Testing how the system handles heavy traffic by simulating many users at once, checking if it stays stable when pushed hard.
- Putting the system into actual use could reveal how easy it is to operate while also testing its durability over time.

14.2 Enhanced Security Validation

Though early tests showed main safety steps - such as scrambling data, verifying users, or checking inputs - do what they're supposed to, later phases need broader checks on protection measures like:

- **Penetration testing:** means hiring good-faith hackers or running software that mimics actual break-in attempts - like dumping code into databases, slipping scripts through web forms, or guessing passwords nonstop.
- **Vulnerability Scanning:** Running expert tools to spot possible flaws in setups or linked components - while checking how pieces interact through automated tests.
- **Security checks:** Getting outside experts to look at how systems are built, go through log files, or examine software code - this helps make sure rules and smart safety methods are actually being followed.

Such checks could give clear proof about how safe the system is while pointing out spots that need fixing.

14.3 Advanced Features and Functionality

To boost what SecureFocus can do and keep up with rivals, upcoming updates might add:

- Custom role setup: Lets you fine-tune access levels with user-defined positions instead of just basic ones like "Owner" or "Member."
- Live chat or calls built right in - helps teammates stay in sync while working on tasks together.
- Smart help for tasks: using habits to guess what to do next or sort jobs by importance.
- A mobile app built just for iPhones along with Android phones will work side by side with the website that adjusts to any screen size.

These features could improve how users interact with the system, while also giving various team setups greater adaptability and stronger functionality.

15. References

- Efimova, D. (2024, July 12). *Comparison of SDLC Models*. Comparison of SDLC Models: How to Choose the Best for Your Project?
<https://startups.epam.com/blog/software-development-models-comparison#:~:text=Advantages>
- Introduction to Software Engineering. (2011). In *Software Engineering* (9th ed., pp. 27-82). Pearson.
- Stevens, G. (2025, February 27). *Why cloud-based productivity suites are a must-have in 2025*. PRODUCTIVITY IN THE WORKPLACE.
<https://www.zoho.com/workplace/articles/cloud-based-productivity-suite.html#:~:text=What%20are%20cloud>
- Boehm, B., & Basili, V. R. (2001). *Software defect reduction top 10 list*. *IEEE Computer*, 34(1), 135–137.
<https://doi.org/10.1109/2.962984>
- ISO/IEC. (2022). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements (ISO/IEC 27001:2022)*. International Organization for Standardization.
- McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley Professional.
- Mead, N. R., & Stehney, T. (2005). *Security quality requirements engineering (SQUARE) methodology*. Carnegie Mellon University, Software Engineering Institute.
- Microsoft. (2005). *The STRIDE threat model*. Microsoft Developer Network.
[https://learn.microsoft.com/en-us/previous-versions/ee823878\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ee823878(v=msdn.10))
- National Institute of Standards and Technology (NIST). (2020). *Security and privacy controls for information systems and organizations (NIST SP 800-53 Rev. 5)*. U.S. Department of Commerce.
- Open Web Application Security Project (OWASP). (2023). *OWASP Top Ten Web Application Security Risks*. OWASP Foundation. <https://owasp.org/www-project-top-ten/>
- OWASP Foundation. (2023). *Authentication and Authorization Cheat Sheet*. OWASP Foundation.
<https://cheatsheetseries.owasp.org/>